

Reviewer Pack — Negative Eigenvalue Count Lower Bounds SOS Degree for Max-CU...

Entry 510fd15279df · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 01:47:11 UTC

Negative Eigenvalue Count Lower Bounds SOS Degree for Max-CUT

Entry ID: 510fd15279df **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 01:47:11 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For any n -variable Max-CUT instance, the minimal SOS degree d required to certify its integrality gap satisfies $d \geq k + 1$, where k is the number of negative eigenvalues of the moment matrix of the degree- d pseudoexpectation.

Rationale (proposer’s reasoning):

The number of negative eigenvalues quantifies the ‘deficiency’ in positive semidefiniteness of the moment matrix, which directly correlates with the need for higher-degree SOS constraints to resolve non-positivity in the polynomial representation of the Max-CUT objective.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f7ee5a5bdfdlab98

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = 1 / A[i][i]
        for j in range(i, n):
            A[i][j] *= factor
        for k in range(i+1, n):
            factor = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
    return [row[:i+1] for row in A]

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0]*n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def is_positive_definite(M):
    n = len(M)
    for i in range(n):
        if M[i][i] <= 0:
            return False
```

```

        factor = 1 / M[i][i]
        for j in range(i, n):
            M[i][j] *= factor
        for k in range(i+1, n):
            factor = M[k][i]
            for j in range(i, n):
                M[k][j] -= factor * M[i][j]
    return True

def max_cut_instance(n):
    # Generate a random Max-CUT instance
    edges = [(random.randint(0, n-1), random.randint(0, n-1)) for _ in range(int(n*(n-1)/2))]
    weights = [random.random() for _ in range(len(edges))]
    return edges, weights

def moment_matrix(instance, degree):
    # Construct the moment matrix of a given degree
    edges, weights = instance
    n = len(set(u for u, v in edges))
    M = [[0] * (n**2) for _ in range(n**2)]
    for u, v in edges:
        idx_u = u * n + v
        idx_v = v * n + u
        M[idx_u][idx_u] += weights[edges.index((u, v))]
        M[idx_u][idx_v] -= weights[edges.index((u, v))]
        M[idx_v][idx_u] -= weights[edges.index((u, v))]
        M[idx_v][idx_v] += weights[edges.index((u, v))]
    return M

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instance = max_cut_instance(n)

    min_sos_degree = float('inf')
    negative_eigenvalues_count = 0

    for d in range(2, 11):
        M = moment_matrix(instance, d)
        eigenvalues = [abs(e) for e in gaussian_elimination(M)[-1]]
        negative_eigenvalues_count += sum(1 for e in eigenvalues if e < 1e-6)

        # Check if the matrix is positive definite
        if not is_positive_definite(M):
            min_sos_degree = d
            break

    conjecture_holds = min_sos_degree >= negative_eigenvalues_count + 1
    counterexample = "" if conjecture_holds else f"min_sos_degree={min_sos_degree}, negative_eigenvalues_count={negative_eigenvalues_count}"

    return {
        "metric_name": "SOS Degree",
        "metric_value": min_sos_degree,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

```

```

    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 50))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ee3b164.py", line 116, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ee3b164.py", line 91, in run_trial
    eigenvalues = [abs(e) for e in gaussian_elimination(M)[-1]]
                      ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ee3b164.py", line 28, in gaussian_elimination
    factor = 1 / A[i][i]
              ~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing data collection. Pre-registered support condition cannot be evaluated without valid results. | next: Debug the gaussian_elimination function to handle singular matrices, then re-run tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	98082
2	propose	ollama_remote	qwen3:8b	0	0	111319
3	propose	ollama_remote	qwen3:8b	0	0	56037
4	propose	ollama_remote	qwen3:8b	0	0	100235
5	preregistration	ollama_remote	qwen3:8b	0	0	24198
6	novelty	ollama_remote	qwen3:8b	0	0	20881
7	novelty	ollama_remote	qwen3:8b	0	0	11933
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14118
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13199
10	judge	ollama_remote	qwen3:8b	0	0	19319

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 469320 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/510fd15279df.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/510fd15279df.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/510fd15279df.tar.gz* (if generated)